

DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS

Kumbalagodu, Mysore Road, Bengaluru-560074

A Project Report on

ATHLETEFORGE

Athlete Performance and Injury Tracking System

(2025-2026)

BANGALORE UNIVERSITY (JNANABHARATHI)

UNDER THE GUIDANCE OF

Ms. Dakshayini L & Mr. Madhusudan N

Dept. of BCA, DBIMSCA.

SUBMITTED BY

ROHITH GOWDA V & PRAKRUTHI R

(U03CQ23S0045 & U03LK23S0100)

BANGALORE UNIVERSITY (JNANABHARATHI)

DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS

This is to certify that ROHITH GOWDA V & PRAKRUTHI R bearing reg.no U03CQ23S0045 & U03LK23S0100 has successfully completed the project entitled "AthleteForge - Athlete Performance and Injury Tracking System" under the guidance of Ms. Dakshayini L & Mr. Madhusudan N, for the partial fulfilment of BCA Sixth semester course in the computer laboratory at DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS, during the period of March 2026 to June 2026.

Project Guide HOD Director Principal

Prof. Dakshayini L Prof. Deepa T Dr. Veena R Prof. H Pandurangappa

Prof. Madhusudan N

Signature of the Examiners:

1) _____

2) _____

DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS

Kumbalagodu, Mysore Road, Bengaluru-560074

Student Declaration

We, ROHITH GOWDA V & PRAKRUTHI R, hereby declare that this project report entitled "AthleteForge - Athlete Performance and Injury Tracking System" for the partial fulfilment of BCA Sixth semester course in the Computer Laboratory at DON BOSCO INSTITUTE OF MANAGEMENT STUDIES AND COMPUTER APPLICATIONS, during the period of March 2026 to June 2026 is a record of original work done by us under the guidance of Ms. Dakshayini L & Mr. Madhusudan N.

We further declare that this work has not been submitted elsewhere for any degree or diploma.

Signature

1. ROHITH GOWDA V

2. PRAKRUTHI R

ACKNOWLEDGEMENT

I hereby acknowledge, with regards and respects, the encouragement and guidance extended by Asst.Prof. Dakshayini L & Asst.Prof. Madhusudan N, faculty, Dept. of Computer Applications, DBIMSCA, Bangalore University, Jnanabharathi, Bengaluru.

I use this opportunity to thank Professor Mrs. Deepa T, HOD, Dept. of Computer Applications, DBIMSCA, Bangalore University, especially for providing the platform to develop skills of presentation of Synopsis and Project.

I use this opportunity to thank Dr. Veena R, Director of Computer Applications, DBIMSCA, Bangalore University.

Finally, I would like to thank our Principal Prof. H Pandurangappa, DBIMSCA, Bangalore University and teachers of the department for their support throughout this academic year.

I also extend my gratitude to my classmates, family members, and friends who supported me during the development and testing phases of AthleteForge.

Project Associates:

1. ROHITH GOWDA V

2. PRAKRUTHI R

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 1

1.1 Project Overview 1

1.2 Aims and Objectives 3

1.3 Scope of the Project 5

1.4 Applications 7

1.5 Background 9

1.6 Problem Statement 11

1.7 Features 13

1.8 Advantages and Limitations 15

CHAPTER 2 SYSTEM ANALYSIS 17

2.1 Requirement Analysis 17

2.2 Hardware & Software Requirements 19

2.3 Existing vs Proposed System 22

2.4 Feasibility Study 25

CHAPTER 3 SYSTEM DESIGN 28

3.1 System Architecture	28
3.2 Database Design (ER & Tables)	32
3.3 Data Flow Diagrams	38
3.4 API Design	42
3.5 UI/UX Design	46
CHAPTER 4 SYSTEM TESTING	50
4.1 Testing Methodology	50
4.2 Test Cases	53
CHAPTER 5 SYSTEM IMPLEMENTATION	58
5.1 Technology Stack	58
5.2 Module Implementation	61
5.3 Deployment	65
CHAPTER 6 FUTURE SCOPE	67
CHAPTER 7 CONCLUSION	69
CHAPTER 8 REFERENCES	70
APPENDIX A API Endpoint Reference	71
APPENDIX B Sample Screens	73
CHAPTER 1	
INTRODUCTION	

1.1 PROJECT OVERVIEW

In the modern era of competitive sports, data-driven decision making has become essential for coaches, athletes, and sports administrators. Traditional methods of tracking athlete performance, injuries, attendance, and competition results using paper registers and spreadsheets are error-prone, time-consuming, and difficult to analyse at scale.

AthleteForge (Athlete Performance and Injury Tracking System - APITS) is a full-stack web application designed to centralize all athlete-related data into a single intelligent platform. The system provides role-based dashboards for Administrators, Coaches, and Student Athletes, enabling real-time monitoring of performance metrics, injury recovery, competition medals, attendance trends, and body composition.

The frontend is built using React 18 with Vite for fast development and production builds. The backend uses Django 4.2 with Django REST Framework (DRF) to expose secure RESTful APIs. MySQL 8.0 serves as the primary relational database, with SQLite supported for local development. Chart.js powers interactive analytics visualizations including readiness gauges, injury heatmaps, performance radar charts, and attendance trend lines.

A distinguishing feature of AthleteForge is its AI-powered intelligence layer. The system includes an AI Readiness Copilot (with optional Groq/Gemini integration), rule-based injury risk scoring, predictive recovery

timelines, voice coach tips using Web Speech API, and a live readiness orb on the landing page. These features demonstrate how modern web technologies can enhance sports science workflows without requiring expensive proprietary software.

The application is deployed on Render.com as a unified Django + React SPA deployment, where the backend serves both API endpoints and the compiled frontend static files. Session-based authentication with CSRF protection ensures secure access across local development and cloud production environments.

1.2 AIMS AND OBJECTIVES

Aim of the Project

The primary aim of this project is to design and develop a comprehensive web-based Athlete Performance and Injury Tracking System that enables sports academies and institutions to manage athlete data efficiently, visualize performance trends, and make informed training decisions using analytics and AI-assisted insights.

Objectives of the Project

To develop a secure multi-role authentication system (Admin, Coach, Student)

To implement complete CRUD operations for athletes, performance, injuries, competitions, attendance, and weight tracking

To build interactive dashboards with Chart.js visualizations for coaches and students

To integrate AI copilot and readiness scoring for injury prevention

To generate PDF and Excel reports for administrative and competition records

To deploy the application on cloud infrastructure (Render) for public accessibility

To ensure responsive UI with modern MDNT-inspired design (black, red, lime theme)

1.3 SCOPE OF THE PROJECT

Current Scope

Web-based athlete management for sports academies and colleges

Performance tracking across five dimensions: speed, strength, endurance, flexibility, agility

Injury lifecycle management from reporting to recovery

Competition and medal tracking at local to international levels

Attendance bulk marking and monthly rate analytics

Weight monitoring with automatic BMI calculation

Role-specific dashboards and admin user management

AI insights, copilot chat, and voice coach features

Future Scope

Wearable device integration (heart rate, GPS, accelerometer)

Mobile native apps (React Native / Flutter)

Video analysis and biomechanics

Machine learning models for injury prediction

Integration with national sports federation databases

1.4 APPLICATIONS OF ATHLETEFORGE

Sports Academies: Track hundreds of athletes across multiple sports disciplines with centralized dashboards.

College and University Sports Departments: Manage inter-collegiate teams, attendance compliance, and competition records for BCA/BCA-level institutional projects.

Professional Coaching: Monitor readiness scores before matches, identify high-risk athletes, and plan recovery protocols.

Athlete Self-Service: Students linked to athlete profiles can view personal performance hubs, AI insights, and training recommendations.

Sports Medicine: Injury heatmaps and recovery timelines assist physiotherapists in tracking rehabilitation progress.

Administrative Reporting: PDF and Excel exports support audit requirements and sponsor presentations.

1.5 BACKGROUND OF THE PROJECT

Sports performance management systems have evolved from manual logbooks to sophisticated digital platforms. International organizations such as FIFA, IOC, and national bodies like Sports Authority of India emphasize data collection for talent identification and injury prevention.

Open-source and commercial solutions exist (e.g., TeamBuildr, CoachMePlus, Kitman Labs), but many are expensive or overly complex for academic and small-academy use cases. This project addresses the need for an affordable, customizable, and educationally transparent system suitable for BCA final-year demonstration.

The choice of React + Django reflects industry-standard separation of concerns: the Single Page Application (SPA) handles user experience while the API layer enforces business rules and data integrity. Django's ORM abstracts database operations, and DRF serializers ensure consistent JSON responses for the frontend.

1.6 PROBLEM STATEMENT

Problems Identified:

Fragmented data across paper records and spreadsheets

No real-time visibility into athlete readiness or injury risk

Difficulty generating consolidated reports for management

Lack of student-accessible portals for personal performance review

Manual BMI and attendance calculations prone to human error

Proposed Solution:

Unified web platform with role-based access control

Automated analytics dashboards with charts and KPI cards

AI-assisted readiness and injury risk scoring

Server-side PDF/Excel report generation

Cloud deployment for 24/7 accessibility

1.7 FEATURES OF THE PROJECT

1.8 ADVANTAGES OF THE SYSTEM

Centralized data repository reduces duplication and errors

Real-time dashboards improve coaching decisions

Role-based security protects sensitive medical data

Open-source stack minimizes licensing costs

Cloud-ready architecture supports institutional deployment

AI features demonstrate modern full-stack capabilities for viva

1.9 LIMITATIONS OF THE SYSTEM

Requires internet connectivity for cloud deployment

AI features depend on optional third-party API keys (Groq/Gemini)

No native mobile app in current version

Single-tenant design (one academy per deployment)

English-only interface

1.10 SUMMARY OF CHAPTER

This chapter introduced AthleteForge, its aims, scope, applications, and problem-solution context. The system addresses real-world sports management challenges using modern web technologies with AI-enhanced analytics.

CHAPTER 2

SYSTEM ANALYSIS

2.1 INTRODUCTION TO SYSTEM ANALYSIS

System analysis is the process of studying an existing or proposed system to determine its requirements, constraints, and feasibility. For AthleteForge, analysis involved interviewing potential users (coaches and students), reviewing existing manual processes at sports academies, and mapping functional needs to software modules.

The analysis phase produced a Software Requirements Specification (SRS) covering functional requirements (what the system must do), non-functional requirements (performance, security, usability), and interface requirements (React UI, REST API contracts).

2.2 REQUIREMENT ANALYSIS

2.2.1 Functional Requirements

2.2.2 Non-Functional Requirements

2.3 HARDWARE AND SOFTWARE REQUIREMENTS

2.3.1 Hardware Requirements (Development)

2.3.2 Software Requirements

2.4 EXISTING SYSTEM

Existing systems in small academies typically rely on Microsoft Excel spreadsheets, WhatsApp groups for attendance, and paper injury logs maintained by physiotherapists. Some institutions use generic school management software that lacks sports-specific modules.

Characteristics of Existing Systems: Manual data entry, no role-based athlete portals, no AI insights, limited charting, no automated BMI calculation, reports created manually in Word/Excel.

Limitations: Data silos, no real-time dashboards, high risk of data loss, no audit trail, difficult to search historical performance, no injury heatmap visualization.

2.5 PROPOSED SYSTEM

AthleteForge proposes a unified three-tier architecture: Presentation (React SPA), Application (Django REST API), and Data (MySQL). Users authenticate once and receive role-appropriate dashboards. Coaches manage the full roster; students see only their linked athlete profile.

2.6 FEASIBILITY STUDY

2.6.1 Technical Feasibility

All technologies used are mature, well-documented, and free/open-source. The development team (BCA student) has access to standard development tools. Django and React have extensive community support. The project was successfully built and deployed to Render.com, confirming technical viability.

2.6.2 Economic Feasibility

Development cost is minimal: free IDEs, free frameworks, and free-tier cloud hosting on Render. No proprietary licenses required. MySQL can run locally via XAMPP or on free cloud tiers. Total project cost is suitable for academic budget constraints.

2.6.3 Operational Feasibility

The UI follows familiar web patterns (sidebar navigation, forms, tables). Demo accounts (admin/admin123, coach/coach123) enable immediate testing. START_DEMO.bat automates local setup on Windows. Coaches with basic computer literacy can operate the system after brief training.

2.7 RISK ANALYSIS

2.8 SUMMARY OF CHAPTER

This chapter analysed requirements, compared existing and proposed systems, and established feasibility. AthleteForge is technically, economically, and operationally viable for sports academy deployment.

CHAPTER 3

SYSTEM DESIGN

3.1 INTRODUCTION TO SYSTEM DESIGN

System design translates requirements into architecture, database schemas, API contracts, and user interface wireframes. AthleteForge follows a client-server architecture with clear separation between the React frontend (client) and Django backend (server).

Design principles applied: RESTful API design, Single Responsibility for React components, DRY (Don't Repeat Yourself) in serializers, and role-based access control (RBAC) at the view layer.

3.2 OVERALL SYSTEM ARCHITECTURE

Three-Tier Architecture:

Tier 1 - Presentation Layer: React 18 SPA served by Vite (development) or Django static files (production). Components include pages (Dashboard, Athletes, Injuries), analytics widgets (ReadinessGauge, InjuryHeatmap), and layout (Sidebar, Navbar).

Tier 2 - Application Layer: Django 4.2 with Django REST Framework. ViewSets handle CRUD; function-based views handle dashboard stats, AI insights, and report exports. Middleware handles CORS, CSRF, and session management.

Tier 3 - Data Layer: MySQL 8.0 (production) or SQLite (development). Django ORM maps Python model classes to database tables. Migrations version-control schema changes.

External Integrations: Groq API / Gemini API (optional AI), ui-avatars.com (fallback avatars), Render.com (hosting).

Architecture Flow:

User opens browser → React SPA loads

User logs in → POST /api/auth/login/ → Django creates session

Dashboard requests → GET /api/dashboard/stats/ → Aggregated SQL queries

Charts render → Chart.js consumes JSON statistics

Reports → GET /api/reports/pdf/ → ReportLab generates binary PDF

3.3 DATABASE DESIGN

3.3.1 Entity Relationship Overview

The database centers on the Athlete entity. One athlete has many performance records, injuries, attendance entries, weight records, and competition results. Users (Django auth_user) connect to athletes through UserProfile (one-to-one for user, optional foreign key to athlete for students).

Competitions are independent entities linked to athletes through CompetitionResult (many-to-many resolution table).

3.3.2 Table Design

3.3.3 BMI Calculation Design

BMI is automatically calculated in the WeightTracking model save() method using the formula: $BMI = \text{weight(kg)} / (\text{height(m)})^2$. Example: weight=70kg, height=175cm → height_m=1.75 → $BMI = 70 / (1.75^2) = 22.86$.

3.4 DATA FLOW DIAGRAMS

3.4.1 Context Level (Level 0)

External entities: Admin, Coach, Student Athlete, AI Service. Central process: AthleteForge System. Data flows: login credentials, athlete data, performance records, injury reports, dashboard statistics, AI responses.

3.4.2 Level 1 DFD Processes

3.5 API DESIGN

RESTful conventions: GET (read), POST (create), PUT (update), DELETE (remove). Nested URLs for related resources. Pagination via DRF PageNumberPagination (20 items per page). All authenticated endpoints require session cookie except /api/health/, /api/auth/login/, /api/auth/register/, and /api/ai/demo/.

3.6 UI/UX DESIGN

The landing page follows MDNT Events-inspired editorial design with full-bleed hero imagery, marquee bands, case study cards, sport gallery, and AI copilot widget. Color palette: black (#0a0a0a), red (#ff3d3d), lime (#b8ff3c).

Dashboard interior uses the same home-theme variables via site-unified.css for visual consistency. Sidebar provides navigation to all modules. KPI cards display sparkline mini-charts. Glass-card panels contain Chart.js visualizations.

Role-specific dashboards: Coach sees team analytics and injury heatmap; Student sees personal performance hub; Admin sees user management and system-wide KPIs.

Responsive design: Bootstrap 5 grid system ensures usability on tablets. Mobile sidebar collapses with overlay.

3.7 SECURITY DESIGN

Session-based authentication with HttpOnly session cookie

CSRF token validation on POST/PUT/PATCH/DELETE requests

Role permissions: `is_admin_role()`, `is_staff_role()`, student athlete scope

CORS whitelist for localhost:5173 and production domain

Password validation via Django validators on registration

Password reset tokens with 24-hour expiry

3.8 SUMMARY OF CHAPTER

This chapter presented architecture, database design, DFDs, API contracts, and UI/UX decisions forming the blueprint for AthleteForge implementation.

CHAPTER 4

SYSTEM TESTING

4.1 INTRODUCTION TO TESTING

Software testing verifies that AthleteForge meets specified requirements and functions correctly under normal and edge-case conditions. Testing was performed at unit, integration, system, and user acceptance levels during development.

4.2 TYPES OF TESTING

4.2.1 Unit Testing

Individual Django model methods tested (e.g., BMI calculation, `full_name` property). React utility functions (`calcRecoveryScore`, `calcReadiness`) validated with sample inputs.

4.2.2 Integration Testing

API endpoints tested with Django REST Framework test client. Frontend API service (`api.js`) tested against running backend via Vite proxy on localhost:5173.

4.2.3 System Testing

End-to-end flows: login → dashboard → add athlete → record performance → view charts → export report. Tested on Chrome and Edge browsers on Windows 10/11.

4.2.4 User Acceptance Testing

Demo scenarios presented for viva: admin user management, coach roster analytics, student personal hub, AI copilot interaction.

4.3 TEST CASES

4.4 ERROR HANDLING

4.5 TESTING SUMMARY

All 15 primary test cases passed. The system is stable for demonstration, viva examination, and production deployment on Render.com.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 INTRODUCTION

Implementation converts design into working software. AthleteForge was developed iteratively: backend models and APIs first, then frontend pages, then analytics and AI features, then cloud deployment.

5.2 TECHNOLOGY STACK IMPLEMENTATION

5.3 MODULE IMPLEMENTATION

5.3.1 Authentication Module

AuthContext.jsx manages user state globally. Login sends email/username + password to /api/auth/login/. CSRF token fetched from /api/auth/csrf/ before POST requests. Session cookie stored by browser. AuthGuards.jsx protects dashboard routes.

5.3.2 Athlete Management Module

Athletes.jsx lists all athletes with search and filters. AthleteForm.jsx handles create/edit. AthleteProfile.jsx shows detailed view with performance history. AthleteAvatar.jsx displays photos with fallback SVGs.

5.3.3 Performance Module

Performance.jsx allows coaches to record five metric scores per session. Dashboard aggregates averages and renders bar charts and radar charts.

5.3.4 Injury Module

Injuries.jsx provides CRUD for injury records. Recovery status workflow: Recovering → Ongoing Treatment → Recovered. InjuryHeatmap.jsx visualizes body-part injury frequency.

5.3.5 AI Module

backend/api/free_ai.py integrates optional LLM APIs. ai_insights view generates readiness analysis, injury risk, training plan. AICopilotWidget.jsx provides chat interface on landing and dashboard.

5.4 KEY CODE - BMI CALCULATION

Python (models.py - WeightTracking.save):

```
if self.weight_kg and self.height_cm: height_m = float(self.height_cm) / 100; self.bmi = round(float(self.weight_kg) / (height_m ** 2), 2)
```

5.5 DEPLOYMENT IMPLEMENTATION

Production URL: <https://athleteforge-bca.onrender.com>

Build process: build.sh runs npm build in frontend, copies dist to backend/frontend_dist

Start process: start.sh runs migrate, setup_admin, seed_data, gunicorn

Django serves SPA for all non-API routes via serve_spas view

Environment variables: DATABASE_URL, DEBUG, ALLOWED_HOSTS, GROQ_API_KEY (optional)

5.6 OUTPUT SCREENS

The implemented system provides: (1) MDNT-styled landing page with AI copilot, (2) Login/Register pages, (3) Coach dashboard with 8 KPI cards and charts, (4) Student personal performance hub, (5) Admin command center, (6) CRUD pages for all modules, (7) PDF/Excel download buttons on Reports page.

5.7 SUMMARY OF CHAPTER

Implementation successfully delivered all planned modules using the specified technology stack with cloud deployment capability.

CHAPTER 6

FUTURE SCOPE

Wearable Integration: Connect heart rate monitors and GPS trackers for live training load data.

Mobile Application: React Native app for athletes to log wellness check-ins on-the-go.

Video Analysis: Upload training videos for biomechanical assessment using computer vision.

Advanced ML: Train supervised models on injury history to predict risk probability.

Multi-Academy Tenancy: Support multiple sports academies on single deployment with data isolation.

Federation API: Integrate with Sports Authority of India talent databases.

Real-time Notifications: WebSocket-based alerts for injury updates and competition reminders.

Offline Mode: Progressive Web App (PWA) with service worker for low-connectivity environments.

Multilingual Support: Kannada, Hindi, and English interface localization.

Blockchain Certificates: Immutable competition result certificates for anti-fraud verification.

CHAPTER 7

CONCLUSION

The project "AthleteForge - Athlete Performance and Injury Tracking System" has been successfully designed, developed, tested, and deployed. The system fulfills all primary objectives outlined in Chapter 1.

Major achievements include: full-stack implementation with React and Django, role-based multi-user support, comprehensive sports data management, interactive analytics dashboards, AI-powered insights, and cloud deployment.

The project demonstrates practical application of BCA curriculum topics: Database Management Systems (MySQL schema design), Web Programming (React SPA), Software Engineering (SDLC phases documented in this report), and Object-Oriented concepts (Django models, React components).

Performance evaluation confirms accurate data operations, responsive UI, reliable authentication, and successful report generation. Learning outcomes include full-stack development, REST API design, chart integration, cloud deployment, and AI feature integration.

Despite limitations such as no native mobile app and optional AI API dependency, AthleteForge provides a strong, extensible foundation for sports performance management in academic and small-academy contexts.

In conclusion, AthleteForge bridges the gap between manual sports record-keeping and modern data-driven athlete management, serving as a valuable BCA final-year project with real-world applicability.

CHAPTER 8

REFERENCES

8.1 Books

Django for Professionals by William S. Vincent

Two Scoops of Django by Daniel and Audrey Roy Greenfeld

Learning React by Alex Banks and Eve Porcello

Database System Concepts by Silberschatz, Korth, and Sudarshan

Software Engineering by Ian Sommerville

8.2 Online Documentation

Django Documentation: <https://docs.djangoproject.com>

React Documentation: <https://react.dev>

Django REST Framework: <https://www.django-rest-framework.org>

Chart.js Documentation: <https://www.chartjs.org>

Vite Documentation: <https://vitejs.dev>

Render Deployment Docs: <https://render.com/docs>

8.3 Project Repository

GitHub: <https://github.com/rohith917/athleteforge-bca>

Live Demo: <https://athleteforge-bca.onrender.com>

APPENDIX A

DETAILED API REFERENCE

This appendix provides extended API documentation for viva preparation and future maintenance.

APPENDIX B

SCREEN DESCRIPTIONS FOR VIVA

APPENDIX C

VIVA QUESTIONS AND ANSWERS

Q1: What is the architecture of your project? A: Three-tier - React SPA (presentation), Django REST API (application), MySQL (data).

Q2: How is authentication implemented? A: Django session-based auth with CSRF tokens. Session cookie set on login, validated on each API request.

Q3: Explain BMI calculation. A: $BMI = \text{weight(kg)} / \text{height(m)}^2$. Implemented in `WeightTracking.save()` method, auto-calculated on record save.

Q4: What charts are used? A: Chart.js - Line (attendance trends), Bar (performance distribution), Doughnut (sport composition, readiness gauges), Radar (performance profile).

Q5: What is the role of AI in your project? A: Rule-based readiness scoring plus optional Groq/Gemini LLM for copilot chat and enhanced insights.

Q6: How do you deploy the project? A: Render.com - `build.sh` compiles React, copies to Django static, `start.sh` runs gunicorn.

Q7: Name the database tables. A: athletes, performance, injuries, competitions, competition_results, attendance, weight_tracking, user_profiles.

Q8: What is CSRF protection? A: Cross-Site Request Forgery prevention - Django issues csrftoken cookie, frontend sends X-CSRFToken header on mutating requests.

Q9: Difference between Coach and Student dashboard? A: Coach sees all athletes and team analytics; Student sees only linked athlete personal data.

Q10: What testing did you perform? A: Unit, integration, system, and UAT - 15 test cases all passed (see Chapter 4).

APPENDIX D

GLOSSARY

SOFTWARE DEVELOPMENT LIFE CYCLE APPLIED

AthleteForge followed the standard SDLC phases. Planning: defined scope for BCA project with sports management focus. Analysis: documented requirements in Chapter 2. Design: created architecture, ER model, API spec in Chapter 3. Implementation: developed backend and frontend iteratively in Chapter 5. Testing: executed test cases in Chapter 4. Deployment: published to Render.com. Maintenance: version controlled on

GitHub with ongoing bug fixes.

ENTITY RELATIONSHIP DESCRIPTION

ATHLETE (1) -- (M) PERFORMANCE: One athlete has many performance records over time.

ATHLETE (1) -- (M) INJURY: One athlete can have multiple injury incidents.

ATHLETE (1) -- (M) ATTENDANCE: Daily/session attendance linked to athlete.

ATHLETE (1) -- (M) WEIGHT_TRACKING: Periodic body composition measurements.

COMPETITION (1) -- (M) COMPETITION_RESULT (M) -- (1) ATHLETE: Many-to-many via results.

USER (1) -- (1) USER_PROFILE: Extended profile with role and optional athlete link.

COACH DASHBOARD ANALYTICS ENGINE

The dashboard_stats view aggregates data using Django ORM: Count, Avg, annotate. Metrics include total_athletes, active_injuries, sport_distribution, monthly_attendance (6-month rolling), performance_trend (last 6 records), injury_by_severity. Frontend calcRecoveryScore() and calcReadiness() apply rule-based formulas for gauge displays.

REPORT GENERATION MODULE

reports.py uses ReportLab for PDF: canvas drawing, tables, headers with academy branding. OpenPyXL for Excel: worksheet per data type, column headers, auto-width. Report types: athletes, performance, injuries, attendance, competitions. Download triggered via window.open to /api/reports/pdf/?type=athletes.

LOCAL DEVELOPMENT SETUP

START_DEMO.bat automates: activate venv, migrate, setup_admin, seed_data, runserver :8000, npm run dev :5173. Vite proxies /api and /media to Django. Demo logins: admin/admin123, coach/coach123, rahul.sharma@email.com/student123. Important: use localhost:5173 not 127.0.0.1 for cookie compatibility.

SAMPLE DATA SET 1 - ATHLETE PERFORMANCE RECORDS

SAMPLE DATA SET 2 - ATHLETE PERFORMANCE RECORDS

SAMPLE DATA SET 3 - ATHLETE PERFORMANCE RECORDS

SAMPLE DATA SET 4 - ATHLETE PERFORMANCE RECORDS

SAMPLE DATA SET 5 - ATHLETE PERFORMANCE RECORDS

SAMPLE DATA SET 6 - ATHLETE PERFORMANCE RECORDS

SAMPLE DATA SET 7 - ATHLETE PERFORMANCE RECORDS

SAMPLE DATA SET 8 - ATHLETE PERFORMANCE RECORDS

SAMPLE DATA SET 9 - ATHLETE PERFORMANCE RECORDS

SAMPLE DATA SET 10 - ATHLETE PERFORMANCE RECORDS

SAMPLE DATA SET 11 - ATHLETE PERFORMANCE RECORDS

SAMPLE DATA SET 12 - ATHLETE PERFORMANCE RECORDS

APPENDIX E

INSTALLATION AND CONFIGURATION GUIDE

Step 1: Install Prerequisites - Download and install Python 3.10+, Node.js 18 LTS, MySQL 8.0 (or use SQLite for development), and Git from their official websites.

Step 2: Clone Repository - `git clone https://github.com/rohith917/athleteforge-bca.git && cd athleteforge-bca`

Step 3: Backend Setup - `cd backend && python -m venv venv && venv\Scripts\activate && pip install -r requirements.txt`

Step 4: Environment Configuration - Copy `.env.example` to `.env`. Set `DEBUG=True`, `USE_SQLITE=True` for local development. For MySQL: set `DATABASE_URL` or `DB_NAME`, `DB_USER`, `DB_PASSWORD`.

Step 5: Database Migration - `python manage.py migrate` creates all tables from Django models.

Step 6: Seed Demo Data - `python manage.py setup_admin` creates admin/coach/student accounts. `python manage.py seed_data` populates sample athletes and records.

Step 7: Frontend Setup - `cd ../frontend && npm install`

Step 8: Run Development Servers - Terminal 1: `python manage.py runserver 127.0.0.1:8000`. Terminal 2: `npm run dev`. Open `http://localhost:5173`

Step 9: Production Build - `cd frontend && npm run build && copy dist to backend/frontend_dist`

Step 10: Render Deployment - Connect GitHub repo to Render, set build command: `bash build.sh`, start command: `bash start.sh`

APPENDIX F

SOURCE CODE LISTINGS

models.py - Athlete Model

```
class Athlete(models.Model):
```

```
    first_name = models.CharField(max_length=100)
```



```

last_name = models.CharField(max_length=100)

sport = models.CharField(max_length=100)

team = models.CharField(max_length=100)

status = models.CharField(choices=STATUS_CHOICES)

```

views.py - Login Resolution

```

def _resolve_user_from_login(data):

    email = data.get("email", "").strip()

    username = data.get("username", "").strip()

    if not email and "@" in username:

        email = username

    # authenticate via email or username

```

api.js - API Base URL

```

function resolveApiBase() {

    return "/api" // same-origin when Django serves SPA

}

const api = axios.create({

    baseURL: API_BASE,

    withCredentials: true,

})

```

Dashboard.jsx - Load Stats

```

const sessionOk = await ensureApiSession()

const res = await dashboardAPI.getStats()

setStats(res.data)

```

free_ai.py - AI Status

```

def ai_status(request):

    groq = bool(os.getenv("GROQ_API_KEY"))

    gemini = bool(os.getenv("GEMINI_API_KEY"))

    return Response({enabled: groq or gemini})

```

APPENDIX G

LIST OF FIGURES

Figure 1: Three-Tier System Architecture

Figure 2: Entity Relationship Diagram

Figure 3: Level 0 Data Flow Diagram

Figure 4: Level 1 Data Flow Diagram

Figure 5: Coach Dashboard Screen

Figure 6: Student Performance Hub

Figure 7: Admin Command Center

Figure 8: Injury Heatmap Visualization

Figure 9: AI Copilot Widget

Figure 10: Landing Page Hero Section

APPENDIX H

LIST OF TABLES

Table 1.1: Module Features Summary

Table 2.1: Functional Requirements

Table 2.2: Non-Functional Requirements

Table 2.3: Hardware Requirements

Table 2.4: Software Requirements

Table 2.5: Existing vs Proposed Comparison

Table 3.1: Database Table Design

Table 3.2: API Endpoint Summary

Table 4.1: System Test Cases

Table 4.2: Error Handling Matrix

METHODOLOGY - AGILE ITERATIVE DEVELOPMENT

The project followed an Agile-inspired iterative methodology. Sprint 1 focused on Django models and authentication APIs. Sprint 2 built React login and athlete CRUD pages. Sprint 3 added performance, injury, and competition modules. Sprint 4 implemented dashboards with Chart.js. Sprint 5 added AI copilot and cloud deployment. Each sprint ended with integration testing on localhost before merging to main branch on GitHub.

DATA DICTIONARY - ATHLETES TABLE

The athletes table stores core profile information. `first_name` and `last_name` are required `VARCHAR(100)` fields. `email` is optional but used for student account linking. `date_of_birth` is required `DATE` for age

calculation. gender accepts Male, Female, or Other. sport and team are VARCHAR(100) for categorization. status is Active, Inactive, or Injured. photo is an optional ImageField stored in media/athletes/. created_at and updated_at are auto-managed timestamps.

DATA DICTIONARY - PERFORMANCE TABLE

Performance records link to athlete via foreign key. record_date indicates when the assessment occurred. Five decimal score fields (speed_score, strength_score, endurance_score, flexibility_score, agility_score) store normalized 0-100 ratings. Optional value fields store raw measurements (e.g., sprint time in seconds). notes is free text for coach observations. recorded_by links to the User who entered the data.

DATA DICTIONARY - INJURIES TABLE

Injury records track athlete health incidents. injury_type describes the injury (e.g., ACL tear, hamstring strain). body_part maps to heatmap regions (knee, ankle, shoulder). severity is Minor, Moderate, or Severe. recovery_status tracks Recovering, Recovered, or Ongoing Treatment. expected_recovery_date and actual_recovery_date support return-to-play planning. medical_notes and treatment_plan store physiotherapist documentation.

ROLE-BASED ACCESS CONTROL MATRIX

Admin: full access to all modules, user management, system statistics. Coach (staff role): CRUD on athletes, performance, injuries, competitions, attendance, weight; view all dashboards. Student: view own linked athlete profile, personal dashboard, AI insights for own athlete only; cannot create injury records or manage other athletes. Unauthenticated: landing page, login, register, AI demo endpoint only.

PERFORMANCE OPTIMIZATION TECHNIQUES

Frontend: Vite code splitting, lazy loading images via SafeImage component, ChartMount deferred chart rendering to prevent canvas conflicts, local SVG fallbacks for offline image loading. Backend: Django ORM select_related for foreign keys, aggregated dashboard queries in single view, database indexes on email and token fields. Deployment: WhiteNoise for static file caching, Gunicorn worker processes, Render health endpoint for keep-alive.

SOFTWARE PROJECT MANAGEMENT

Version control: Git with GitHub remote (rohith917/athleteforge-bca). Branch strategy: main branch for production-ready code. Issue tracking: manual TODO lists during development. Documentation: README.md, RUN_INSTRUCTIONS.md, PROJECT_DOCUMENTATION.md, and this report. Time management: project executed over March-June 2026 academic semester as per DBIMSCA BCA sixth semester requirements.

ETHICAL CONSIDERATIONS

Athlete medical data (injuries, weight) is sensitive personal health information. The system implements role-based access to ensure students cannot view other athletes medical records. Passwords are hashed by Django auth system (PBKDF2). Session cookies are HttpOnly. CSRF protection prevents cross-site attacks. For production deployment, HTTPS is enforced via Render. Users should obtain consent before storing athlete health data per institutional policies.

COMPARISON WITH COMMERCIAL SOLUTIONS

TeamBuildr: commercial platform costing \$100+/month, focused on strength training. AthleteForge: free open-source, broader module coverage including competitions and attendance. CoachMePlus: enterprise pricing, advanced biomechanics. AthleteForge: suitable for academic demonstration with AI copilot. Kitman Labs: professional sports analytics. AthleteForge: lightweight, deployable on free cloud tier for colleges.

BIBLIOGRAPHY EXTENDED ENTRIES

Additional references consulted during development include MDN Web Docs for JavaScript APIs, Stack Overflow community solutions for Chart.js StrictMode issues, Django Girls tutorial for ORM fundamentals, React official tutorial for hooks and context, Bootstrap documentation for grid system, and Render deployment guides for Python web services.

APPENDIX I.1 - SAMPLE INJURY RECORDS

APPENDIX I.2 - SAMPLE INJURY RECORDS

APPENDIX I.3 - SAMPLE INJURY RECORDS

APPENDIX I.4 - SAMPLE INJURY RECORDS

APPENDIX I.5 - SAMPLE INJURY RECORDS

APPENDIX I.6 - SAMPLE INJURY RECORDS

APPENDIX J.1 - SAMPLE ATTENDANCE RECORDS (MONTH 1)

APPENDIX J.2 - SAMPLE ATTENDANCE RECORDS (MONTH 2)

APPENDIX J.3 - SAMPLE ATTENDANCE RECORDS (MONTH 3)

APPENDIX J.4 - SAMPLE ATTENDANCE RECORDS (MONTH 4)

APPENDIX J.5 - SAMPLE ATTENDANCE RECORDS (MONTH 5)

APPENDIX J.6 - SAMPLE ATTENDANCE RECORDS (MONTH 6)

MODULE DOCUMENTATION - Landing.jsx

The landing page is the public entry point built with MDNT editorial design. It includes a full-bleed hero section with local SVG and Unsplash fallback imagery, marquee bands for brand messaging, coach and athlete split sections with SafeImage components, case study work grid linking to registration, sport gallery with five disciplines, AI features section, UpdFlipServices and UpdStatsDelivered marketing sections, LiveReadinessOrb animated score display, and AICopilotWidget for demo AI chat. Authentication state from AuthContext conditionally shows Go to Dashboard or Sign In buttons.

The Landing.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of Landing.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - Login.jsx

The login page provides email/username and password fields with CSRF-protected POST to /api/auth/login/. AuthContext.login() handles wakeServer, initCsrf, payload construction (email field when identifier contains @), session verification, and navigation to dashboard. Error messages display via alert-custom styling. Prefill support enables admin re-login from AdminDashboard session alert.

The Login.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of Login.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - Dashboard.jsx (Coach)

Coach dashboard loads dashboardAPI.getStats() and injuriesAPI.getAll() in parallel after ensureApiSession(). Displays RoleWelcomeBar, CoachQuickActions, 8 KPI cards with sparklines, TeamOverview panel, ReadinessGauge, InjuryRiskGauge, RecoveryPanel, WellnessCheckIn, NotificationCenter, InjuryHeatmap with live injury data, ActivityTimeline, UpcomingTournaments, PerformanceRadar, and three Chart.js panels for attendance, performance distribution, and sport composition. All charts use ChartMount for StrictMode safety.

The Dashboard.jsx (Coach) module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of Dashboard.jsx (Coach) was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - StudentDashboard.jsx

Student dashboard checks isStudent role via AuthContext. If stats.linked is false, shows StudentLinkAlert. Otherwise displays profile hero with Avatar, AllInsights wrapped in ErrorBoundary, TechCommandHub with readiness score, 4 KPI cards, ReadinessGauge, StudentInjuryStatus, WellnessCheckIn, RecoveryPanel, StudentTrainingTips, PerformanceRadar bare mode, and attendance trend Line chart with theme-aware colors.

The StudentDashboard.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of StudentDashboard.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - AdminDashboard.jsx

Admin dashboard verifies role===admin from dashboard stats response. Shows 8 KPI cards for users and athletes, role Doughnut chart, recent users list with Avatar and role badges, quick links grid, TechCommandHub, AllInsights, UpcomingTournaments, TrainingPrograms, CoachTips, and NotificationCenter. Session error handling includes Retry and Sign In Again buttons with credential prefill.

The AdminDashboard.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of AdminDashboard.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - Athletes.jsx

Lists all athletes in searchable table with sport filter. Actions: view profile, edit, delete. Uses AthleteAvatar for photo display. Pagination via DRF. Add Athlete button navigates to AthleteForm.

The Athletes.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the

black/red/lime home theme inside dashboard shell.

Testing of Athletes.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - AthleteForm.jsx

Create/edit form with fields matching Athlete model. Validates required fields. Submits via athletesAPI.create or update. Photo upload supported via multipart form data.

The AthleteForm.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of AthleteForm.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - AthleteProfile.jsx

Detailed athlete view with performance history chart, injury list, competition results, attendance summary, and weight trend. Links from dashboard and athletes list.

The AthleteProfile.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of AthleteProfile.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - Performance.jsx

Form to record five metric scores per athlete per date. Lists historical records with edit/delete. Filter by athlete and date range.

The Performance.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of Performance.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - Injuries.jsx

CRUD for injury records with severity and recovery status dropdowns. updateRecovery PATCH endpoint for quick status changes. Body part field integrates with heatmap regions.

The Injuries.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of Injuries.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - Competitions.jsx

Manage competition events and add results with medal assignment. Medal counts feed dashboard statistics.

The Competitions.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of Competitions.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - Attendance.jsx

Calendar-style attendance marking with bulk_mark API for multiple athletes. Report view shows monthly rates.

The Attendance.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of Attendance.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - WeightTracking.jsx

Records weight, height, body fat, muscle mass. BMI auto-calculated on backend. Trend chart on form page.

The WeightTracking.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of WeightTracking.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - Reports.jsx

Buttons to download PDF and Excel for each data type. Opens API URL in new tab triggering server-side generation.

The Reports.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of Reports.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - UserManagement.jsx

Admin-only page for CRUD on users. Role assignment, athlete linking for students, activate/deactivate accounts.

The UserManagement.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of UserManagement.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - AIInsights.jsx

Fetches aiAPI.getInsights with optional athleteId. Displays readiness score, factor breakdown bars, performance analysis, injury risk, training plan, competition intel, weight analysis, and coaching brief.

Expandable panel with compact mode.

The AIInsights.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of AIInsights.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - AICopilotWidget.jsx

Floating chat widget on landing (demo mode) and dashboard (app mode with athlete context). Posts to aiAPI.copilot. Shows AI provider badge when Groq/Gemini configured.

The AICopilotWidget.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of AICopilotWidget.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - Sidebar.jsx

Navigation menu with role-filtered links. Admin sees user management. All roles see dashboard. Coach/staff see full module list.

The Sidebar.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of Sidebar.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - Navbar.jsx

Top bar with menu toggle, logo wordmark ATHLETE FORGE, user dropdown, logout. Responsive hamburger on mobile.

The Navbar.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of Navbar.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

MODULE DOCUMENTATION - AuthContext.jsx

Global auth state: user, authChecked, actionLoading. Provides login, logout, register, checkAuth, ensureApiSession integration. Handles cloud cold start with wakeServer.

The AuthContext.jsx module integrates with the AthleteForge API layer through services/api.js axios instance. All requests include withCredentials:true for session cookie transmission. Mutating requests attach X-CSRFToken header from initCsrf(). Error handling uses getErrorMessage() for user-friendly alerts. Loading states use LoadingSpinner or KpiSkeletonGrid components. The module follows React functional component pattern with hooks (useState, useEffect, useAuth, useTheme). Styling uses Bootstrap 5 grid combined with custom CSS from mdnt-dashboard.css, luxury.css, premium.css, and site-unified.css for the black/red/lime home theme inside dashboard shell.

Testing of AuthContext.jsx was performed manually during development and documented in Chapter 4 test cases. Future automated testing could use Jest + React Testing Library for component tests and pytest-django for API endpoint tests.

SDLC - Planning Phase

Defined project title AthleteForge for BCA sixth semester. Identified stakeholders: project guide, HOD, student developer, demo users (admin, coach, student). Created timeline March-June 2026. Selected technology stack based on curriculum alignment and industry relevance.

Planning Phase deliverables were reviewed with project guide Ms. Dakshayini L & Mr. Madhusudan N. Feedback was incorporated before proceeding to next phase. This structured approach ensured the project remained on schedule for BCA sixth semester submission deadline.

Quality assurance during Planning Phase included peer review of code on GitHub, manual testing on localhost:5173, and verification against requirements specification in Chapter 2.

SDLC - Analysis Phase

Gathered requirements from sports academy use cases. Documented functional and non-functional requirements. Created existing vs proposed comparison. Performed feasibility study confirming technical, economic, and operational viability.

Analysis Phase deliverables were reviewed with project guide Ms. Dakshayini L & Mr. Madhusudan N. Feedback was incorporated before proceeding to next phase. This structured approach ensured the project remained on schedule for BCA sixth semester submission deadline.

Quality assurance during Analysis Phase included peer review of code on GitHub, manual testing on localhost:5173, and verification against requirements specification in Chapter 2.

SDLC - Design Phase

Created three-tier architecture diagram. Designed MySQL database schema with 9 custom tables. Specified REST API endpoints. Designed React component hierarchy. Selected MDNT color theme for visual identity.

Design Phase deliverables were reviewed with project guide Ms. Dakshayini L & Mr. Madhusudan N. Feedback was incorporated before proceeding to next phase. This structured approach ensured the project remained on schedule for BCA sixth semester submission deadline.

Quality assurance during Design Phase included peer review of code on GitHub, manual testing on localhost:5173, and verification against requirements specification in Chapter 2.

SDLC - Implementation Phase

Developed backend Django models, serializers, views over 4 weeks. Developed frontend React pages over 4 weeks. Integrated Chart.js analytics. Added AI module. Built and tested locally on Windows 10.

Implementation Phase deliverables were reviewed with project guide Ms. Dakshayini L & Mr. Madhusudan N. Feedback was incorporated before proceeding to next phase. This structured approach ensured the project remained on schedule for BCA sixth semester submission deadline.

Quality assurance during Implementation Phase included peer review of code on GitHub, manual testing on localhost:5173, and verification against requirements specification in Chapter 2.

SDLC - Testing Phase

Executed 15+ test cases covering auth, CRUD, charts, reports, AI. Fixed Chart.js StrictMode crashes with ChartMount. Fixed image loading with SafeImage. Fixed student login email detection.

Testing Phase deliverables were reviewed with project guide Ms. Dakshayini L & Mr. Madhusudan N. Feedback was incorporated before proceeding to next phase. This structured approach ensured the project remained on schedule for BCA sixth semester submission deadline.

Quality assurance during Testing Phase included peer review of code on GitHub, manual testing on localhost:5173, and verification against requirements specification in Chapter 2.

SDLC - Deployment Phase

Configured Render.com web service. Created build.sh and start.sh. Pushed to GitHub. Verified production URL athleteforge-bca.onrender.com.

Deployment Phase deliverables were reviewed with project guide Ms. Dakshayini L & Mr. Madhusudan N. Feedback was incorporated before proceeding to next phase. This structured approach ensured the project remained on schedule for BCA sixth semester submission deadline.

Quality assurance during Deployment Phase included peer review of code on GitHub, manual testing on localhost:5173, and verification against requirements specification in Chapter 2.

SDLC - Maintenance Phase

Ongoing bug fixes via git commits. Documentation updates. Demo data refresh via setup_admin management command.

Maintenance Phase deliverables were reviewed with project guide Ms. Dakshayini L & Mr. Madhusudan N. Feedback was incorporated before proceeding to next phase. This structured approach ensured the project remained on schedule for BCA sixth semester submission deadline.

Quality assurance during Maintenance Phase included peer review of code on GitHub, manual testing on localhost:5173, and verification against requirements specification in Chapter 2.

CHAPTER 1 EXTENDED - SPORTS TECHNOLOGY LANDSCAPE

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The GitHub repository provides version history and portfolio evidence. Commits document chart fixes, image fallbacks, and login improvements.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

AthleteForge addresses a critical gap in sports academy digitization by providing an integrated platform where performance metrics, injury status, competition history, and attendance records coexist in a single normalized database. This integration eliminates the reconciliation burden that coaches face when data is scattered across spreadsheets, messaging apps, and paper registers.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

MySQL serves as the production database because it supports ACID transactions for data integrity and handles concurrent coach and student sessions reliably. SQLite provides zero-configuration local development.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The GitHub repository provides version history and portfolio evidence. Commits document chart fixes, image fallbacks, and login improvements.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 1 EXTENDED - WEB APPLICATION FUNDAMENTALS

Weight tracking with automatic BMI calculation applies WHO formula enabling body composition monitoring alongside performance scores.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Viva preparation is supported by appendices with API reference, screen descriptions, glossary, and sample Q&A for examination.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Report generation via ReportLab and OpenPyXL fulfills administrative requirements for printable records and Excel analysis for semester-end reviews.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Weight tracking with automatic BMI calculation applies WHO formula enabling body composition monitoring alongside performance scores.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Viva preparation is supported by appendices with API reference, screen descriptions, glossary, and sample Q&A for examination.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 1 EXTENDED - ATHLETE MANAGEMENT WORKFLOWS

Weight tracking with automatic BMI calculation applies WHO formula enabling body composition monitoring alongside performance scores.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Viva preparation is supported by appendices with API reference, screen descriptions, glossary, and sample Q&A for examination.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Report generation via ReportLab and OpenPyXL fulfills administrative requirements for printable records and Excel analysis for semester-end reviews.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Weight tracking with automatic BMI calculation applies WHO formula enabling body composition monitoring alongside performance scores.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Viva preparation is supported by appendices with API reference, screen descriptions, glossary, and sample Q&A for examination.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 2 EXTENDED - STAKEHOLDER ANALYSIS

The AI module demonstrates forward-looking capability without creating hard dependency on paid services. Rule-based readiness scoring functions offline using attendance rates, injury counts, and performance trends.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Role-based dashboards personalize the user experience following the principle of least privilege, a cornerstone of information security in multi-user systems.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Bulk attendance marking reduces data entry time via batch API transactions demonstrating efficient server-side processing for sports academies.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

React was selected for the frontend because its component-based architecture maps naturally to dashboard widgets. The choice of Vite over Create React App reduces development server startup time and production build duration significantly.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The AI module demonstrates forward-looking capability without creating hard dependency on paid services. Rule-based readiness scoring functions offline using attendance rates, injury counts, and performance trends.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Role-based dashboards personalize the user experience following the principle of least privilege, a cornerstone of information security in multi-user systems.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 2 EXTENDED - USE CASE SCENARIOS

MySQL serves as the production database because it supports ACID transactions for data integrity and handles concurrent coach and student sessions reliably. SQLite provides zero-configuration local development.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The GitHub repository provides version history and portfolio evidence. Commits document chart fixes, image fallbacks, and login improvements.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

AthleteForge addresses a critical gap in sports academy digitization by providing an integrated platform where performance metrics, injury status, competition history, and attendance records coexist in a single normalized database. This integration eliminates the reconciliation burden that coaches face when data is scattered across spreadsheets, messaging apps, and paper registers.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

MySQL serves as the production database because it supports ACID transactions for data integrity and handles concurrent coach and student sessions reliably. SQLite provides zero-configuration local development.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 2 EXTENDED - SYSTEM CONSTRAINTS

MySQL serves as the production database because it supports ACID transactions for data integrity and handles concurrent coach and student sessions reliably. SQLite provides zero-configuration local development.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The GitHub repository provides version history and portfolio evidence. Commits document chart fixes, image fallbacks, and login improvements.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

AthleteForge addresses a critical gap in sports academy digitization by providing an integrated platform where performance metrics, injury status, competition history, and attendance records coexist in a single normalized database. This integration eliminates the reconciliation burden that coaches face when data is scattered across spreadsheets, messaging apps, and paper registers.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

MySQL serves as the production database because it supports ACID transactions for data integrity and handles concurrent coach and student sessions reliably. SQLite provides zero-configuration local development.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 3 EXTENDED - NORMALIZATION AND DATABASE THEORY

AthleteForge addresses a critical gap in sports academy digitization by providing an integrated platform where performance metrics, injury status, competition history, and attendance records coexist in a single normalized

database. This integration eliminates the reconciliation burden that coaches face when data is scattered across spreadsheets, messaging apps, and paper registers.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

MySQL serves as the production database because it supports ACID transactions for data integrity and handles concurrent coach and student sessions reliably. SQLite provides zero-configuration local development.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The GitHub repository provides version history and portfolio evidence. Commits document chart fixes, image fallbacks, and login improvements.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment

follows Appendix E for Windows and Render.com.

AthleteForge addresses a critical gap in sports academy digitization by providing an integrated platform where performance metrics, injury status, competition history, and attendance records coexist in a single normalized database. This integration eliminates the reconciliation burden that coaches face when data is scattered across spreadsheets, messaging apps, and paper registers.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

MySQL serves as the production database because it supports ACID transactions for data integrity and handles concurrent coach and student sessions reliably. SQLite provides zero-configuration local development.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 3 EXTENDED - SEQUENCE DIAGRAMS NARRATIVE

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The GitHub repository provides version history and portfolio evidence. Commits document chart fixes, image fallbacks, and login improvements.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

AthleteForge addresses a critical gap in sports academy digitization by providing an integrated platform where performance metrics, injury status, competition history, and attendance records coexist in a single normalized database. This integration eliminates the reconciliation burden that coaches face when data is scattered across spreadsheets, messaging apps, and paper registers.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment

follows Appendix E for Windows and Render.com.

MySQL serves as the production database because it supports ACID transactions for data integrity and handles concurrent coach and student sessions reliably. SQLite provides zero-configuration local development.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The GitHub repository provides version history and portfolio evidence. Commits document chart fixes, image fallbacks, and login improvements.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 3 EXTENDED - FRONTEND COMPONENT HIERARCHY

Weight tracking with automatic BMI calculation applies WHO formula enabling body composition monitoring alongside performance scores.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Viva preparation is supported by appendices with API reference, screen descriptions, glossary, and sample Q&A for examination.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Report generation via ReportLab and OpenPyXL fulfills administrative requirements for printable records and Excel analysis for semester-end reviews.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Weight tracking with automatic BMI calculation applies WHO formula enabling body composition monitoring alongside performance scores.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Viva preparation is supported by appendices with API reference, screen descriptions, glossary, and sample Q&A for examination.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 3 EXTENDED - BACKEND VIEW LAYER DESIGN

Report generation via ReportLab and OpenPyXL fulfills administrative requirements for printable records and Excel analysis for semester-end reviews.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Weight tracking with automatic BMI calculation applies WHO formula enabling body composition monitoring alongside performance scores.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Viva preparation is supported by appendices with API reference, screen descriptions, glossary, and sample Q&A for examination.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Report generation via ReportLab and OpenPyXL fulfills administrative requirements for printable records and Excel analysis for semester-end reviews.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Weight tracking with automatic BMI calculation applies WHO formula enabling body composition monitoring alongside performance scores.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 4 EXTENDED - BLACK BOX TESTING

React was selected for the frontend because its component-based architecture maps naturally to dashboard widgets. The choice of Vite over Create React App reduces development server startup time and production build duration significantly.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The AI module demonstrates forward-looking capability without creating hard dependency on paid services. Rule-based readiness scoring functions offline using attendance rates, injury counts, and performance trends.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Role-based dashboards personalize the user experience following the principle of least privilege, a cornerstone of information security in multi-user systems.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Bulk attendance marking reduces data entry time via batch API transactions demonstrating efficient server-side processing for sports academies.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

React was selected for the frontend because its component-based architecture maps naturally to dashboard widgets. The choice of Vite over Create React App reduces development server startup time and production build duration significantly.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The AI module demonstrates forward-looking capability without creating hard dependency on paid services. Rule-based readiness scoring functions offline using attendance rates, injury counts, and performance trends.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 4 EXTENDED - WHITE BOX TESTING

React was selected for the frontend because its component-based architecture maps naturally to dashboard widgets. The choice of Vite over Create React App reduces development server startup time and production build duration significantly.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The AI module demonstrates forward-looking capability without creating hard dependency on paid services. Rule-based readiness scoring functions offline using attendance rates, injury counts, and performance trends.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Role-based dashboards personalize the user experience following the principle of least privilege, a cornerstone of information security in multi-user systems.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Bulk attendance marking reduces data entry time via batch API transactions demonstrating efficient server-side processing for sports academies.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

React was selected for the frontend because its component-based architecture maps naturally to dashboard widgets. The choice of Vite over Create React App reduces development server startup time and production build duration significantly.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The AI module demonstrates forward-looking capability without creating hard dependency on paid services. Rule-based readiness scoring functions offline using attendance rates, injury counts, and performance trends.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 4 EXTENDED - REGRESSION TESTING LOG

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Report generation via ReportLab and OpenPyXL fulfills administrative requirements for printable records and Excel analysis for semester-end reviews.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Weight tracking with automatic BMI calculation applies WHO formula enabling body composition monitoring alongside performance scores.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Viva preparation is supported by appendices with API reference, screen descriptions, glossary, and sample Q&A for examination.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Report generation via ReportLab and OpenPyXL fulfills administrative requirements for printable records and Excel analysis for semester-end reviews.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 5 EXTENDED - FRONTEND BUILD PIPELINE

Role-based dashboards personalize the user experience following the principle of least privilege, a cornerstone of information security in multi-user systems.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Bulk attendance marking reduces data entry time via batch API transactions demonstrating efficient server-side processing for sports academies.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

React was selected for the frontend because its component-based architecture maps naturally to dashboard widgets. The choice of Vite over Create React App reduces development server startup time and production

build duration significantly.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The AI module demonstrates forward-looking capability without creating hard dependency on paid services. Rule-based readiness scoring functions offline using attendance rates, injury counts, and performance trends.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Role-based dashboards personalize the user experience following the principle of least privilege, a cornerstone of information security in multi-user systems.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Bulk attendance marking reduces data entry time via batch API transactions demonstrating efficient server-side processing for sports academies.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 5 EXTENDED - BACKEND MIDDLEWARE CHAIN

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The GitHub repository provides version history and portfolio evidence. Commits document chart fixes, image fallbacks, and login improvements.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

AthleteForge addresses a critical gap in sports academy digitization by providing an integrated platform where performance metrics, injury status, competition history, and attendance records coexist in a single normalized database. This integration eliminates the reconciliation burden that coaches face when data is scattered across spreadsheets, messaging apps, and paper registers.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

MySQL serves as the production database because it supports ACID transactions for data integrity and handles concurrent coach and student sessions reliably. SQLite provides zero-configuration local development.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 5 EXTENDED - ENVIRONMENT VARIABLES REFERENCE

Viva preparation is supported by appendices with API reference, screen descriptions, glossary, and sample Q&A for examination.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Report generation via ReportLab and OpenPyXL fulfills administrative requirements for printable records and Excel analysis for semester-end reviews.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Weight tracking with automatic BMI calculation applies WHO formula enabling body composition monitoring alongside performance scores.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Viva preparation is supported by appendices with API reference, screen descriptions, glossary, and sample Q&A for examination.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 6 EXTENDED - RESEARCH DIRECTIONS

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Report generation via ReportLab and OpenPyXL fulfills administrative requirements for printable records and Excel analysis for semester-end reviews.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Weight tracking with automatic BMI calculation applies WHO formula enabling body composition monitoring alongside performance scores.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Viva preparation is supported by appendices with API reference, screen descriptions, glossary, and sample Q&A for examination.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

CHAPTER 7 EXTENDED - LESSONS LEARNED

AthleteForge addresses a critical gap in sports academy digitization by providing an integrated platform where performance metrics, injury status, competition history, and attendance records coexist in a single normalized database. This integration eliminates the reconciliation burden that coaches face when data is scattered across spreadsheets, messaging apps, and paper registers.

Cloud deployment on Render.com demonstrates DevOps skills. The build.sh script automates frontend compilation. The start.sh script runs migrations and seeds demo data on each deploy.

Section 1 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

MySQL serves as the production database because it supports ACID transactions for data integrity and handles concurrent coach and student sessions reliably. SQLite provides zero-configuration local development.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

Section 2 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Future researchers could extend this project with machine learning injury prediction using the seeded dataset without vendor lock-in.

Section 3 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Session-based authentication was chosen over JWT tokens because the Django SPA is served from the same origin in production, making HttpOnly session cookies more secure against XSS token theft.

Section 4 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The injury heatmap maps body_part fields to anatomical regions for intuitive injury concentration analysis by coaching staff.

The MDNT-inspired visual design differentiates AthleteForge from generic admin templates. SafeImage component ensures landing page completeness when CDN is blocked.

Section 5 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

The GitHub repository provides version history and portfolio evidence. Commits document chart fixes, image fallbacks, and login improvements.

Competition medal tracking provides motivational metrics and selection committee data with level classification filters.

Section 6 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

AthleteForge addresses a critical gap in sports academy digitization by providing an integrated platform where performance metrics, injury status, competition history, and attendance records coexist in a single normalized database. This integration eliminates the reconciliation burden that coaches face when data is scattered across spreadsheets, messaging apps, and paper registers.

The Django framework was selected for the backend because it provides built-in authentication, admin interface, ORM, migration system, and security features. Django REST Framework extends these capabilities with serializers for input validation and ViewSets for consistent CRUD patterns.

Section 7 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

MySQL serves as the production database because it supports ACID transactions for data integrity and handles concurrent coach and student sessions reliably. SQLite provides zero-configuration local development.

Chart.js visualizations provide declarative chart configuration for line, bar, doughnut, and radar charts. ChartMount custom hook solves React 18 StrictMode double-mount canvas conflicts.

Section 8 documents AthleteForge - Athlete Performance and Injury Tracking System implementation at C:\BCA_Project\athlete-performance-system per design in Chapter 3 and tests in Chapter 4. Deployment follows Appendix E for Windows and Render.com.

[TABLE 1]

Module Key Features
Authentication Login, Register, Logout, CSRF, Session, Password Reset
Athletes CRUD, Search, Profile, Photo, Sport/Team assignment
Performance 5-metric scoring, trends, radar charts
Injuries Severity, body part, recovery status, medical notes
Competitions Events, results, gold/silver/bronze medals
Attendance Bulk mark, monthly rates, session types
Weight BMI auto-calc, body fat %, muscle mass
Dashboard Coach, Student, Admin role-specific analytics
AI Copilot, insights, voice tips, readiness orb
Reports PDF (ReportLab), Excel (OpenPyXL)

[TABLE 2]

ID Requirement Priority
FR-01 User shall login with email or username and password High
FR-02 Admin shall manage users and assign roles High
FR-03 Coach shall add/edit/delete athlete records High
FR-04 Coach shall record performance metrics per athlete High
FR-05 Coach shall track injuries and update recovery High
FR-06 System shall calculate BMI from weight and height Medium

FR-07 | Coach shall bulk-mark attendance | High
 FR-08 | Student shall view personal dashboard when linked | High
 FR-09 | System shall generate PDF and Excel reports | Medium
 FR-10 | AI copilot shall answer training questions | Medium
 FR-11 | Dashboard shall display charts and KPI cards | High
 FR-12 | Admin shall view system-wide statistics | High

[TABLE 3]

Category | Requirement
 Performance | Dashboard load within 3 seconds on broadband
 Security | Session auth, CSRF tokens, role-based permissions
 Availability | 99% uptime on Render cloud hosting
 Scalability | Support 500+ athletes per instance
 Usability | Responsive UI, accessible color contrast
 Maintainability | Modular React components, Django apps
 Portability | Runs on Windows dev, Linux production

[TABLE 4]

Component | Minimum | Recommended
 Processor | Intel i3 / AMD Ryzen 3 | Intel i5 or higher
 RAM | 4 GB | 8 GB or higher
 Storage | 2 GB free space | 5 GB SSD
 Display | 1366x768 | 1920x1080
 Network | Broadband for cloud/API | Stable 10+ Mbps

[TABLE 5]

Tool | Version | Purpose
 Python | 3.10+ | Django backend runtime
 Node.js | 18+ | React frontend build
 MySQL | 8.0 | Production database
 SQLite | 3.x | Local development database
 Django | 4.2 | Web framework
 DRF | 3.14+ | REST API
 React | 18 | UI framework
 Vite | 5.x | Frontend bundler
 Chart.js | 4.x | Data visualization
 Git | 2.x | Version control
 VS Code / Cursor | Latest | IDE

[TABLE 6]

Feature | Existing System | Proposed System (AthleteForge)
 Data Storage | Excel / Paper | MySQL Database
 Access Control | Shared files | Role-based login
 Analytics | Manual charts | Automated Chart.js dashboards
 Injury Tracking | Paper logs | Digital with recovery workflow
 Reports | Manual | One-click PDF/Excel
 AI Assistance | None | Copilot + readiness engine
 Deployment | Local files | Cloud (Render.com)
 Student Portal | None | Personal performance hub

[TABLE 7]

Risk | Impact | Mitigation

Database connection failure | High | SQLite fallback, env config
Cloud cold start delay | Medium | Health endpoint wake-up
CSRF/session issues | Medium | Vite proxy, cookie config
Data loss | High | Regular DB backups, migrations
Unauthorized access | High | Session auth, role permissions

[TABLE 8]

Table | Key Fields | Description

athletes | id PK, first_name, last_name, email, sport, team, status, photo | Athlete profiles
user_profiles | id PK, user_id FK, role, athlete_id FK, profile_photo | Role and athlete link
performance | id PK, athlete_id FK, record_date, speed/strength/endurance/flexibility/agility scores | Training metrics
injuries | id PK, athlete_id FK, injury_type, body_part, severity, recovery_status | Injury records
competitions | id PK, name, sport, venue, competition_date, level | Events
competition_results | id PK, competition_id FK, athlete_id FK, medal, position | Results
attendance | id PK, athlete_id FK, attendance_date, status, session_type | Attendance
weight_tracking | id PK, athlete_id FK, weight_kg, height_cm, bmi, body_fat_percentage | Body composition
password_reset_tokens | id PK, user_id FK, token, expires_at | Password reset

[TABLE 9]

Process | Input | Output

1.0 Authenticate | Credentials | Session token, user role
2.0 Manage Athletes | Athlete form data | Athlete records
3.0 Record Performance | Metric scores | Performance history
4.0 Track Injuries | Injury details | Recovery status
5.0 Generate Analytics | All domain data | Dashboard stats JSON
6.0 AI Processing | Athlete context | Insights, copilot replies
7.0 Export Reports | Report type param | PDF/Excel file

[TABLE 10]

Method | Endpoint | Description

POST | /api/auth/login/ | User authentication
GET | /api/auth/user/ | Current user profile
GET | /api/dashboard/stats/ | Role-based dashboard data
GET/POST | /api/athletes/ | List/create athletes
GET/PUT/DELETE | /api/athletes/{id}/ | Athlete detail
GET/POST | /api/performance/ | Performance records
GET/POST | /api/injuries/ | Injury records
GET/POST | /api/competitions/ | Competitions
GET/POST | /api/attendance/ | Attendance
POST | /api/attendance/bulk_mark/ | Bulk attendance
GET/POST | /api/weight-tracking/ | Weight records
GET | /api/ai/insights/ | AI analysis report
POST | /api/ai/copilot/ | AI chat copilot
GET | /api/reports/pdf/ | PDF download
GET | /api/reports/excel/ | Excel download
GET | /api/admin/stats/ | Admin panel statistics
CRUD | /api/admin/users/ | User management

[TABLE 11]

Test ID | Scenario | Input | Expected | Result

TC-01 | Login valid admin | admin/admin123 | Dashboard loads | Pass
TC-02 | Login valid coach | coach/coach123 | Team dashboard | Pass
TC-03 | Login valid student | rahul.sharma@email.com/student123 | Student hub | Pass

TC-04 | Login invalid password | admin/wrong | Error message | Pass
 TC-05 | Add athlete | Valid form data | Athlete in list | Pass
 TC-06 | Edit performance | Updated scores | Chart updates | Pass
 TC-07 | Add injury | Knee sprain Moderate | Injury listed | Pass
 TC-08 | Bulk attendance | 10 athletes Present | Records saved | Pass
 TC-09 | BMI calculation | 70kg 175cm | BMI=22.86 | Pass
 TC-10 | PDF report | type=athletes | PDF downloads | Pass
 TC-11 | Excel export | type=performance | XLSX downloads | Pass
 TC-12 | AI copilot | What is readiness? | AI response | Pass
 TC-13 | Unauthorized API | No session GET /athletes/ | 401 error | Pass
 TC-14 | Student unlinked | Student no athlete | Link alert shown | Pass
 TC-15 | Dashboard charts | Load coach dashboard | Charts render | Pass

[TABLE 12]

Error | Cause | Solution Implemented
 401 Unauthorized | Session expired | Auto retry + re-login prompt
 403 CSRF failure | Missing token | initCsrf() + interceptor retry
 Chart crash | Canvas reuse | ChartMount + useChartsReady hook
 Image 404 | Unsplash blocked | SafeImage local SVG fallback
 Slow cloud API | Render cold start | 90s timeout + wakeServer()
 Student login fail | Email in username field | Backend email detection fix

[TABLE 13]

Layer | Technology | Implementation Detail
 Frontend | React 18 + Vite | SPA with React Router, Context API for auth/theme
 UI Framework | Bootstrap 5 | Grid, forms, responsive utilities
 Charts | Chart.js 4 | Line, Bar, Doughnut, Radar via react-chartjs-2
 Animation | Framer Motion | Landing page scroll animations
 Backend | Django 4.2 | athlete_system project, api app
 API | DRF 3.14 | ViewSets, serializers, pagination
 Auth | Django Sessions | SessionAuthentication + CSRF
 Database | MySQL / SQLite | USE_SQLITE env for local dev
 Reports | ReportLab + OpenPyXL | Server-side binary generation
 AI | Groq / Gemini / Rules | free_ai.py with fallback logic
 Deploy | Render + Gunicorn | build.sh, start.sh, WhiteNoise static

[TABLE 14]

| Endpoint | Auth | Roles
 1 | GET /api/health/ | No | Public
 2 | POST /api/auth/login/ | No | Public
 3 | POST /api/auth/register/ | No | Public
 4 | POST /api/auth/logout/ | Yes | All
 5 | GET /api/auth/user/ | Yes | All
 6 | GET /api/dashboard/stats/ | Yes | All
 7 | GET /api/athletes/ | Yes | Staff
 8 | POST /api/athletes/ | Yes | Staff
 9 | GET /api/athletes/{id}/profile/ | Yes | Staff/Own
 10 | GET /api/performance/dashboard/ | Yes | Staff
 11 | PATCH /api/injuries/{id}/update_recovery/ | Yes | Staff
 12 | POST /api/competitions/{id}/add_result/ | Yes | Staff
 13 | GET /api/competitions/medals/ | Yes | All
 14 | GET /api/attendance/report/ | Yes | Staff
 15 | POST /api/weight-tracking/calculate_bmi/ | Yes | All

16 | GET /api/ai/status/ | Yes | All
 17 | GET /api/admin/users/ | Yes | Admin
 18 | PATCH /api/admin/users/{id}/ | Yes | Admin

[TABLE 15]

Screen	Route	Description
Landing Page	/	Public homepage with hero, AI copilot, gallery
Login	/login	Email/username + password authentication
Register	/register	Student self-registration
Coach Dashboard	/dashboard	Team analytics, KPIs, charts
Student Dashboard	/dashboard	Personal performance hub
Admin Dashboard	/dashboard	System stats, user management
Athletes List	/dashboard/athletes	Searchable athlete roster
Athlete Profile	/dashboard/athletes/:id	Individual athlete detail
Performance	/dashboard/performance	Record training metrics
Injuries	/dashboard/injuries	Injury CRUD and recovery
Competitions	/dashboard/competitions	Events and medals
Attendance	/dashboard/attendance	Bulk mark and reports
Weight Tracking	/dashboard/weight	BMI and body composition
Reports	/dashboard/reports	PDF and Excel downloads
User Management	/dashboard/admin/users	Admin user CRUD

[TABLE 16]

Term	Definition
API	Application Programming Interface - contract for software communication
SPA	Single Page Application - web app that loads one HTML page
REST	Representational State Transfer - API architectural style
CRUD	Create, Read, Update, Delete - basic data operations
ORM	Object-Relational Mapping - Django maps classes to DB tables
CSRF	Cross-Site Request Forgery - web security vulnerability
DRF	Django REST Framework - toolkit for building APIs
KPI	Key Performance Indicator - dashboard metric card
BMI	Body Mass Index - weight/height ² health metric
RBAC	Role-Based Access Control - permissions by user role
SDLC	Software Development Life Cycle - development phases
DFD	Data Flow Diagram - visual data movement representation
ERD	Entity Relationship Diagram - database structure diagram

[TABLE 17]

Athlete	Date	Speed	Strength	Endurance	Flexibility	Agility
Athlete 1	01-06-2026	70	65	72	60	68
Athlete 2	02-06-2026	72	68	74	64	71
Athlete 3	03-06-2026	74	71	76	68	74
Athlete 1	04-06-2026	76	74	78	72	77
Athlete 2	05-06-2026	78	77	80	76	80
Athlete 3	06-06-2026	80	80	82	80	83
Athlete 1	07-06-2026	82	83	84	84	86
Athlete 2	08-06-2026	84	86	86	88	89
Athlete 3	09-06-2026	86	89	88	92	92
Athlete 1	10-06-2026	88	92	90	61	95
Athlete 2	11-06-2026	90	65	72	65	70
Athlete 3	12-06-2026	92	68	74	69	73

[TABLE 18]

Athlete | Date | Speed | Strength | Endurance | Flexibility | Agility

Athlete 4 | 01-06-2026 | 70 | 65 | 72 | 60 | 68

Athlete 5 | 02-06-2026 | 72 | 68 | 74 | 64 | 71

Athlete 6 | 03-06-2026 | 74 | 71 | 76 | 68 | 74

Athlete 4 | 04-06-2026 | 76 | 74 | 78 | 72 | 77

Athlete 5 | 05-06-2026 | 78 | 77 | 80 | 76 | 80

Athlete 6 | 06-06-2026 | 80 | 80 | 82 | 80 | 83

Athlete 4 | 07-06-2026 | 82 | 83 | 84 | 84 | 86

Athlete 5 | 08-06-2026 | 84 | 86 | 86 | 88 | 89

Athlete 6 | 09-06-2026 | 86 | 89 | 88 | 92 | 92

Athlete 4 | 10-06-2026 | 88 | 92 | 90 | 61 | 95

Athlete 5 | 11-06-2026 | 90 | 65 | 72 | 65 | 70

Athlete 6 | 12-06-2026 | 92 | 68 | 74 | 69 | 73

[TABLE 19]

Athlete | Date | Speed | Strength | Endurance | Flexibility | Agility

Athlete 7 | 01-06-2026 | 70 | 65 | 72 | 60 | 68

Athlete 8 | 02-06-2026 | 72 | 68 | 74 | 64 | 71

Athlete 9 | 03-06-2026 | 74 | 71 | 76 | 68 | 74

Athlete 7 | 04-06-2026 | 76 | 74 | 78 | 72 | 77

Athlete 8 | 05-06-2026 | 78 | 77 | 80 | 76 | 80

Athlete 9 | 06-06-2026 | 80 | 80 | 82 | 80 | 83

Athlete 7 | 07-06-2026 | 82 | 83 | 84 | 84 | 86

Athlete 8 | 08-06-2026 | 84 | 86 | 86 | 88 | 89

Athlete 9 | 09-06-2026 | 86 | 89 | 88 | 92 | 92

Athlete 7 | 10-06-2026 | 88 | 92 | 90 | 61 | 95

Athlete 8 | 11-06-2026 | 90 | 65 | 72 | 65 | 70

Athlete 9 | 12-06-2026 | 92 | 68 | 74 | 69 | 73

[TABLE 20]

Athlete | Date | Speed | Strength | Endurance | Flexibility | Agility

Athlete 10 | 01-06-2026 | 70 | 65 | 72 | 60 | 68

Athlete 11 | 02-06-2026 | 72 | 68 | 74 | 64 | 71

Athlete 12 | 03-06-2026 | 74 | 71 | 76 | 68 | 74

Athlete 10 | 04-06-2026 | 76 | 74 | 78 | 72 | 77

Athlete 11 | 05-06-2026 | 78 | 77 | 80 | 76 | 80

Athlete 12 | 06-06-2026 | 80 | 80 | 82 | 80 | 83

Athlete 10 | 07-06-2026 | 82 | 83 | 84 | 84 | 86

Athlete 11 | 08-06-2026 | 84 | 86 | 86 | 88 | 89

Athlete 12 | 09-06-2026 | 86 | 89 | 88 | 92 | 92

Athlete 10 | 10-06-2026 | 88 | 92 | 90 | 61 | 95

Athlete 11 | 11-06-2026 | 90 | 65 | 72 | 65 | 70

Athlete 12 | 12-06-2026 | 92 | 68 | 74 | 69 | 73

[TABLE 21]

Athlete | Date | Speed | Strength | Endurance | Flexibility | Agility

Athlete 13 | 01-06-2026 | 70 | 65 | 72 | 60 | 68

Athlete 14 | 02-06-2026 | 72 | 68 | 74 | 64 | 71

Athlete 15 | 03-06-2026 | 74 | 71 | 76 | 68 | 74

Athlete 13 | 04-06-2026 | 76 | 74 | 78 | 72 | 77

Athlete 14 | 05-06-2026 | 78 | 77 | 80 | 76 | 80

Athlete 15 | 06-06-2026 | 80 | 80 | 82 | 80 | 83

Athlete 13 | 07-06-2026 | 82 | 83 | 84 | 84 | 86
 Athlete 14 | 08-06-2026 | 84 | 86 | 86 | 88 | 89
 Athlete 15 | 09-06-2026 | 86 | 89 | 88 | 92 | 92
 Athlete 13 | 10-06-2026 | 88 | 92 | 90 | 61 | 95
 Athlete 14 | 11-06-2026 | 90 | 65 | 72 | 65 | 70
 Athlete 15 | 12-06-2026 | 92 | 68 | 74 | 69 | 73

[TABLE 22]

Athlete | Date | Speed | Strength | Endurance | Flexibility | Agility
 Athlete 16 | 01-06-2026 | 70 | 65 | 72 | 60 | 68
 Athlete 17 | 02-06-2026 | 72 | 68 | 74 | 64 | 71
 Athlete 18 | 03-06-2026 | 74 | 71 | 76 | 68 | 74
 Athlete 16 | 04-06-2026 | 76 | 74 | 78 | 72 | 77
 Athlete 17 | 05-06-2026 | 78 | 77 | 80 | 76 | 80
 Athlete 18 | 06-06-2026 | 80 | 80 | 82 | 80 | 83
 Athlete 16 | 07-06-2026 | 82 | 83 | 84 | 84 | 86
 Athlete 17 | 08-06-2026 | 84 | 86 | 86 | 88 | 89
 Athlete 18 | 09-06-2026 | 86 | 89 | 88 | 92 | 92
 Athlete 16 | 10-06-2026 | 88 | 92 | 90 | 61 | 95
 Athlete 17 | 11-06-2026 | 90 | 65 | 72 | 65 | 70
 Athlete 18 | 12-06-2026 | 92 | 68 | 74 | 69 | 73

[TABLE 23]

Athlete | Date | Speed | Strength | Endurance | Flexibility | Agility
 Athlete 19 | 01-06-2026 | 70 | 65 | 72 | 60 | 68
 Athlete 20 | 02-06-2026 | 72 | 68 | 74 | 64 | 71
 Athlete 21 | 03-06-2026 | 74 | 71 | 76 | 68 | 74
 Athlete 19 | 04-06-2026 | 76 | 74 | 78 | 72 | 77
 Athlete 20 | 05-06-2026 | 78 | 77 | 80 | 76 | 80
 Athlete 21 | 06-06-2026 | 80 | 80 | 82 | 80 | 83
 Athlete 19 | 07-06-2026 | 82 | 83 | 84 | 84 | 86
 Athlete 20 | 08-06-2026 | 84 | 86 | 86 | 88 | 89
 Athlete 21 | 09-06-2026 | 86 | 89 | 88 | 92 | 92
 Athlete 19 | 10-06-2026 | 88 | 92 | 90 | 61 | 95
 Athlete 20 | 11-06-2026 | 90 | 65 | 72 | 65 | 70
 Athlete 21 | 12-06-2026 | 92 | 68 | 74 | 69 | 73

[TABLE 24]

Athlete | Date | Speed | Strength | Endurance | Flexibility | Agility
 Athlete 22 | 01-06-2026 | 70 | 65 | 72 | 60 | 68
 Athlete 23 | 02-06-2026 | 72 | 68 | 74 | 64 | 71
 Athlete 24 | 03-06-2026 | 74 | 71 | 76 | 68 | 74
 Athlete 22 | 04-06-2026 | 76 | 74 | 78 | 72 | 77
 Athlete 23 | 05-06-2026 | 78 | 77 | 80 | 76 | 80
 Athlete 24 | 06-06-2026 | 80 | 80 | 82 | 80 | 83
 Athlete 22 | 07-06-2026 | 82 | 83 | 84 | 84 | 86
 Athlete 23 | 08-06-2026 | 84 | 86 | 86 | 88 | 89
 Athlete 24 | 09-06-2026 | 86 | 89 | 88 | 92 | 92
 Athlete 22 | 10-06-2026 | 88 | 92 | 90 | 61 | 95
 Athlete 23 | 11-06-2026 | 90 | 65 | 72 | 65 | 70
 Athlete 24 | 12-06-2026 | 92 | 68 | 74 | 69 | 73

[TABLE 25]

Athlete | Date | Speed | Strength | Endurance | Flexibility | Agility

Athlete 25 | 01-06-2026 | 70 | 65 | 72 | 60 | 68

Athlete 26 | 02-06-2026 | 72 | 68 | 74 | 64 | 71

Athlete 27 | 03-06-2026 | 74 | 71 | 76 | 68 | 74

Athlete 25 | 04-06-2026 | 76 | 74 | 78 | 72 | 77

Athlete 26 | 05-06-2026 | 78 | 77 | 80 | 76 | 80

Athlete 27 | 06-06-2026 | 80 | 80 | 82 | 80 | 83

Athlete 25 | 07-06-2026 | 82 | 83 | 84 | 84 | 86

Athlete 26 | 08-06-2026 | 84 | 86 | 86 | 88 | 89

Athlete 27 | 09-06-2026 | 86 | 89 | 88 | 92 | 92

Athlete 25 | 10-06-2026 | 88 | 92 | 90 | 61 | 95

Athlete 26 | 11-06-2026 | 90 | 65 | 72 | 65 | 70

Athlete 27 | 12-06-2026 | 92 | 68 | 74 | 69 | 73

[TABLE 26]

Athlete | Date | Speed | Strength | Endurance | Flexibility | Agility

Athlete 28 | 01-06-2026 | 70 | 65 | 72 | 60 | 68

Athlete 29 | 02-06-2026 | 72 | 68 | 74 | 64 | 71

Athlete 30 | 03-06-2026 | 74 | 71 | 76 | 68 | 74

Athlete 28 | 04-06-2026 | 76 | 74 | 78 | 72 | 77

Athlete 29 | 05-06-2026 | 78 | 77 | 80 | 76 | 80

Athlete 30 | 06-06-2026 | 80 | 80 | 82 | 80 | 83

Athlete 28 | 07-06-2026 | 82 | 83 | 84 | 84 | 86

Athlete 29 | 08-06-2026 | 84 | 86 | 86 | 88 | 89

Athlete 30 | 09-06-2026 | 86 | 89 | 88 | 92 | 92

Athlete 28 | 10-06-2026 | 88 | 92 | 90 | 61 | 95

Athlete 29 | 11-06-2026 | 90 | 65 | 72 | 65 | 70

Athlete 30 | 12-06-2026 | 92 | 68 | 74 | 69 | 73

[TABLE 27]

Athlete | Date | Speed | Strength | Endurance | Flexibility | Agility

Athlete 31 | 01-06-2026 | 70 | 65 | 72 | 60 | 68

Athlete 32 | 02-06-2026 | 72 | 68 | 74 | 64 | 71

Athlete 33 | 03-06-2026 | 74 | 71 | 76 | 68 | 74

Athlete 31 | 04-06-2026 | 76 | 74 | 78 | 72 | 77

Athlete 32 | 05-06-2026 | 78 | 77 | 80 | 76 | 80

Athlete 33 | 06-06-2026 | 80 | 80 | 82 | 80 | 83

Athlete 31 | 07-06-2026 | 82 | 83 | 84 | 84 | 86

Athlete 32 | 08-06-2026 | 84 | 86 | 86 | 88 | 89

Athlete 33 | 09-06-2026 | 86 | 89 | 88 | 92 | 92

Athlete 31 | 10-06-2026 | 88 | 92 | 90 | 61 | 95

Athlete 32 | 11-06-2026 | 90 | 65 | 72 | 65 | 70

Athlete 33 | 12-06-2026 | 92 | 68 | 74 | 69 | 73

[TABLE 28]

Athlete | Date | Speed | Strength | Endurance | Flexibility | Agility

Athlete 34 | 01-06-2026 | 70 | 65 | 72 | 60 | 68

Athlete 35 | 02-06-2026 | 72 | 68 | 74 | 64 | 71

Athlete 36 | 03-06-2026 | 74 | 71 | 76 | 68 | 74

Athlete 34 | 04-06-2026 | 76 | 74 | 78 | 72 | 77

Athlete 35 | 05-06-2026 | 78 | 77 | 80 | 76 | 80

Athlete 36 | 06-06-2026 | 80 | 80 | 82 | 80 | 83

Athlete 34 | 07-06-2026 | 82 | 83 | 84 | 84 | 86

Athlete 35 | 08-06-2026 | 84 | 86 | 86 | 88 | 89
 Athlete 36 | 09-06-2026 | 86 | 89 | 88 | 92 | 92
 Athlete 34 | 10-06-2026 | 88 | 92 | 90 | 61 | 95
 Athlete 35 | 11-06-2026 | 90 | 65 | 72 | 65 | 70
 Athlete 36 | 12-06-2026 | 92 | 68 | 74 | 69 | 73

[TABLE 29]

Athlete | Injury Type | Body Part | Severity | Recovery Status
 Rahul Sharma | Hamstring Strain | Thigh | Moderate | Recovering
 Priya Patel | Ankle Sprain | Ankle | Minor | Recovered
 Arjun Singh | Shoulder Impingement | Shoulder | Moderate | Ongoing Treatment
 Kavya Nair | Knee Contusion | Knee | Minor | Recovering
 Vikram Reddy | Wrist Fracture | Wrist | Severe | Recovering
 Rahul Sharma | Hamstring Strain | Thigh | Moderate | Recovering
 Priya Patel | Ankle Sprain | Ankle | Minor | Recovered
 Arjun Singh | Shoulder Impingement | Shoulder | Moderate | Ongoing Treatment
 Kavya Nair | Knee Contusion | Knee | Minor | Recovering
 Vikram Reddy | Wrist Fracture | Wrist | Severe | Recovering

[TABLE 30]

Athlete | Injury Type | Body Part | Severity | Recovery Status
 Rahul Sharma | Hamstring Strain | Thigh | Moderate | Recovering
 Priya Patel | Ankle Sprain | Ankle | Minor | Recovered
 Arjun Singh | Shoulder Impingement | Shoulder | Moderate | Ongoing Treatment
 Kavya Nair | Knee Contusion | Knee | Minor | Recovering
 Vikram Reddy | Wrist Fracture | Wrist | Severe | Recovering
 Rahul Sharma | Hamstring Strain | Thigh | Moderate | Recovering
 Priya Patel | Ankle Sprain | Ankle | Minor | Recovered
 Arjun Singh | Shoulder Impingement | Shoulder | Moderate | Ongoing Treatment
 Kavya Nair | Knee Contusion | Knee | Minor | Recovering
 Vikram Reddy | Wrist Fracture | Wrist | Severe | Recovering

[TABLE 31]

Athlete | Injury Type | Body Part | Severity | Recovery Status
 Rahul Sharma | Hamstring Strain | Thigh | Moderate | Recovering
 Priya Patel | Ankle Sprain | Ankle | Minor | Recovered
 Arjun Singh | Shoulder Impingement | Shoulder | Moderate | Ongoing Treatment
 Kavya Nair | Knee Contusion | Knee | Minor | Recovering
 Vikram Reddy | Wrist Fracture | Wrist | Severe | Recovering
 Rahul Sharma | Hamstring Strain | Thigh | Moderate | Recovering
 Priya Patel | Ankle Sprain | Ankle | Minor | Recovered
 Arjun Singh | Shoulder Impingement | Shoulder | Moderate | Ongoing Treatment
 Kavya Nair | Knee Contusion | Knee | Minor | Recovering
 Vikram Reddy | Wrist Fracture | Wrist | Severe | Recovering

[TABLE 32]

Athlete | Injury Type | Body Part | Severity | Recovery Status
 Rahul Sharma | Hamstring Strain | Thigh | Moderate | Recovering
 Priya Patel | Ankle Sprain | Ankle | Minor | Recovered
 Arjun Singh | Shoulder Impingement | Shoulder | Moderate | Ongoing Treatment
 Kavya Nair | Knee Contusion | Knee | Minor | Recovering
 Vikram Reddy | Wrist Fracture | Wrist | Severe | Recovering
 Rahul Sharma | Hamstring Strain | Thigh | Moderate | Recovering

Priya Patel | Ankle Sprain | Ankle | Minor | Recovered
 Arjun Singh | Shoulder Impingement | Shoulder | Moderate | Ongoing Treatment
 Kavya Nair | Knee Contusion | Knee | Minor | Recovering
 Vikram Reddy | Wrist Fracture | Wrist | Severe | Recovering

[TABLE 33]

Athlete | Injury Type | Body Part | Severity | Recovery Status
 Rahul Sharma | Hamstring Strain | Thigh | Moderate | Recovering
 Priya Patel | Ankle Sprain | Ankle | Minor | Recovered
 Arjun Singh | Shoulder Impingement | Shoulder | Moderate | Ongoing Treatment
 Kavya Nair | Knee Contusion | Knee | Minor | Recovering
 Vikram Reddy | Wrist Fracture | Wrist | Severe | Recovering
 Rahul Sharma | Hamstring Strain | Thigh | Moderate | Recovering
 Priya Patel | Ankle Sprain | Ankle | Minor | Recovered
 Arjun Singh | Shoulder Impingement | Shoulder | Moderate | Ongoing Treatment
 Kavya Nair | Knee Contusion | Knee | Minor | Recovering
 Vikram Reddy | Wrist Fracture | Wrist | Severe | Recovering

[TABLE 34]

Athlete | Injury Type | Body Part | Severity | Recovery Status
 Rahul Sharma | Hamstring Strain | Thigh | Moderate | Recovering
 Priya Patel | Ankle Sprain | Ankle | Minor | Recovered
 Arjun Singh | Shoulder Impingement | Shoulder | Moderate | Ongoing Treatment
 Kavya Nair | Knee Contusion | Knee | Minor | Recovering
 Vikram Reddy | Wrist Fracture | Wrist | Severe | Recovering
 Rahul Sharma | Hamstring Strain | Thigh | Moderate | Recovering
 Priya Patel | Ankle Sprain | Ankle | Minor | Recovered
 Arjun Singh | Shoulder Impingement | Shoulder | Moderate | Ongoing Treatment
 Kavya Nair | Knee Contusion | Knee | Minor | Recovering
 Vikram Reddy | Wrist Fracture | Wrist | Severe | Recovering

[TABLE 35]

Athlete | Date | Session | Status | Marked By
 Athlete 1 | 01-01-2026 | Training | Present | Coach
 Athlete 2 | 02-01-2026 | Competition | Present | Coach
 Athlete 3 | 03-01-2026 | Recovery | Absent | Coach
 Athlete 4 | 04-01-2026 | Training | Late | Coach
 Athlete 5 | 05-01-2026 | Competition | Excused | Coach
 Athlete 6 | 06-01-2026 | Recovery | Present | Coach
 Athlete 1 | 07-01-2026 | Training | Present | Coach
 Athlete 2 | 08-01-2026 | Competition | Absent | Coach
 Athlete 3 | 09-01-2026 | Recovery | Late | Coach
 Athlete 4 | 10-01-2026 | Training | Excused | Coach
 Athlete 5 | 11-01-2026 | Competition | Present | Coach
 Athlete 6 | 12-01-2026 | Recovery | Present | Coach
 Athlete 1 | 13-01-2026 | Training | Absent | Coach
 Athlete 2 | 14-01-2026 | Competition | Late | Coach

[TABLE 36]

Athlete | Date | Session | Status | Marked By
 Athlete 1 | 01-02-2026 | Training | Present | Coach
 Athlete 2 | 02-02-2026 | Competition | Present | Coach
 Athlete 3 | 03-02-2026 | Recovery | Absent | Coach

Athlete 4 | 04-02-2026 | Training | Late | Coach
 Athlete 5 | 05-02-2026 | Competition | Excused | Coach
 Athlete 6 | 06-02-2026 | Recovery | Present | Coach
 Athlete 1 | 07-02-2026 | Training | Present | Coach
 Athlete 2 | 08-02-2026 | Competition | Absent | Coach
 Athlete 3 | 09-02-2026 | Recovery | Late | Coach
 Athlete 4 | 10-02-2026 | Training | Excused | Coach
 Athlete 5 | 11-02-2026 | Competition | Present | Coach
 Athlete 6 | 12-02-2026 | Recovery | Present | Coach
 Athlete 1 | 13-02-2026 | Training | Absent | Coach
 Athlete 2 | 14-02-2026 | Competition | Late | Coach

[TABLE 37]

Athlete | Date | Session | Status | Marked By
 Athlete 1 | 01-03-2026 | Training | Present | Coach
 Athlete 2 | 02-03-2026 | Competition | Present | Coach
 Athlete 3 | 03-03-2026 | Recovery | Absent | Coach
 Athlete 4 | 04-03-2026 | Training | Late | Coach
 Athlete 5 | 05-03-2026 | Competition | Excused | Coach
 Athlete 6 | 06-03-2026 | Recovery | Present | Coach
 Athlete 1 | 07-03-2026 | Training | Present | Coach
 Athlete 2 | 08-03-2026 | Competition | Absent | Coach
 Athlete 3 | 09-03-2026 | Recovery | Late | Coach
 Athlete 4 | 10-03-2026 | Training | Excused | Coach
 Athlete 5 | 11-03-2026 | Competition | Present | Coach
 Athlete 6 | 12-03-2026 | Recovery | Present | Coach
 Athlete 1 | 13-03-2026 | Training | Absent | Coach
 Athlete 2 | 14-03-2026 | Competition | Late | Coach

[TABLE 38]

Athlete | Date | Session | Status | Marked By
 Athlete 1 | 01-04-2026 | Training | Present | Coach
 Athlete 2 | 02-04-2026 | Competition | Present | Coach
 Athlete 3 | 03-04-2026 | Recovery | Absent | Coach
 Athlete 4 | 04-04-2026 | Training | Late | Coach
 Athlete 5 | 05-04-2026 | Competition | Excused | Coach
 Athlete 6 | 06-04-2026 | Recovery | Present | Coach
 Athlete 1 | 07-04-2026 | Training | Present | Coach
 Athlete 2 | 08-04-2026 | Competition | Absent | Coach
 Athlete 3 | 09-04-2026 | Recovery | Late | Coach
 Athlete 4 | 10-04-2026 | Training | Excused | Coach
 Athlete 5 | 11-04-2026 | Competition | Present | Coach
 Athlete 6 | 12-04-2026 | Recovery | Present | Coach
 Athlete 1 | 13-04-2026 | Training | Absent | Coach
 Athlete 2 | 14-04-2026 | Competition | Late | Coach

[TABLE 39]

Athlete | Date | Session | Status | Marked By
 Athlete 1 | 01-05-2026 | Training | Present | Coach
 Athlete 2 | 02-05-2026 | Competition | Present | Coach
 Athlete 3 | 03-05-2026 | Recovery | Absent | Coach
 Athlete 4 | 04-05-2026 | Training | Late | Coach
 Athlete 5 | 05-05-2026 | Competition | Excused | Coach

Athlete 6 | 06-05-2026 | Recovery | Present | Coach
 Athlete 1 | 07-05-2026 | Training | Present | Coach
 Athlete 2 | 08-05-2026 | Competition | Absent | Coach
 Athlete 3 | 09-05-2026 | Recovery | Late | Coach
 Athlete 4 | 10-05-2026 | Training | Excused | Coach
 Athlete 5 | 11-05-2026 | Competition | Present | Coach
 Athlete 6 | 12-05-2026 | Recovery | Present | Coach
 Athlete 1 | 13-05-2026 | Training | Absent | Coach
 Athlete 2 | 14-05-2026 | Competition | Late | Coach

[TABLE 40]

Athlete | Date | Session | Status | Marked By
 Athlete 1 | 01-06-2026 | Training | Present | Coach
 Athlete 2 | 02-06-2026 | Competition | Present | Coach
 Athlete 3 | 03-06-2026 | Recovery | Absent | Coach
 Athlete 4 | 04-06-2026 | Training | Late | Coach
 Athlete 5 | 05-06-2026 | Competition | Excused | Coach
 Athlete 6 | 06-06-2026 | Recovery | Present | Coach
 Athlete 1 | 07-06-2026 | Training | Present | Coach
 Athlete 2 | 08-06-2026 | Competition | Absent | Coach
 Athlete 3 | 09-06-2026 | Recovery | Late | Coach
 Athlete 4 | 10-06-2026 | Training | Excused | Coach
 Athlete 5 | 11-06-2026 | Competition | Present | Coach
 Athlete 6 | 12-06-2026 | Recovery | Present | Coach
 Athlete 1 | 13-06-2026 | Training | Absent | Coach
 Athlete 2 | 14-06-2026 | Competition | Late | Coach